JOINS

Q1. Basic INNER JOIN

List all sales showing: sale_id, sale_date, customer full_name, and total_amount. Order by sale_date ascending.

Q2. Three-table JOIN

Show every sale with: sale_id, customer full_name, employee full name (first + last), shop name, and total_amount. Order by total_amount descending.

Q3. LEFT JOIN — find missing links

List all customers and how many purchases they have made. Include customers who have never made a purchase (show 0 for them). Show: customer full_name, city, loyalty_tier, purchase_count.

Q4. LEFT JOIN + IS NULL — find non-buyers

Find all customers who have NEVER made a purchase. Show only their names and the city they are from.

Q5. Self JOIN — employee and manager

Show a list of every employee with their manager's name. Include employees who have no manager (show 'No Manager' for those). Columns: employee_name, job_title, manager_name.

Hint: Use COALESCE to replace NULL manager names with 'No Manager'.

Q6. JOIN with filter

Show all sales made via M-Pesa, with the customer's name, shop name, sale date, and amount. Order by sale_date.

Q7. JOIN across four tables

List all sale items showing: sale_id, sale_date, customer full_name, product name, quantity, and line_total. Order by sale_date, then line_total descending.

Q8. JOIN with aggregation

For each shop, show: shop_name, category, number of employees, and average employee salary (round to 2 decimal places). Include shops that have no employees. Order by average salary descending.

Q9. JOIN with date filter

Show all deliveries received in January 2024. Include: supplier_name, product_name, shop_name, delivery_date, qty_delivered, cost_per_unit, and total delivery cost (qty_delivered × cost_per_unit).

Q10. Challenge JOIN

Show each product's full sales history: product_name, shop_name (where it's sold from), sale_date, customer name, quantity sold, and line_total. For products that have never been sold, still show them with NULL for sale columns. Order by product_name, sale_date

Subqueries

Q1. Scalar subquery - compare to average

Find all employees whose salary is above the average salary of all employees. Show: full name, job_title, salary, and the average salary as a column next to each row.

Q2. Scalar subquery in SELECT

Show each sale_id, total_amount, and a column called 'vs_average' that shows how much the sale is above or below the average sale amount (positive = above, negative = below). Round to 2 decimal places.

Q3. List subquery with IN

Find all employees who work in Electronics shops. Use a subquery to get the shop IDs first, then use IN to filter employees. Show: full name, job_title, salary.

Q4. List subquery with NOT IN

Find all products that have NEVER been sold. Use NOT IN with a subquery on sale_items. Show: product_name, category, unit_price, stock_qty.

Q5. FROM subquery

Using a subquery in the FROM clause, first calculate total revenue per shop. Then from that result, show only shops with total revenue above KES 50,000.

Q6. Subquery with MAX

Find the customer who has spent the most in total across all their sales. Show their full_name, city, loyalty_tier, and total amount spent.

Q7. EXISTS subquery

Find all shops that have received at least one supplier delivery. Use EXISTS. Show shop_name and category.

Q8. NOT EXISTS subquery

Find all shops that have NEVER received any supplier delivery. Use NOT EXISTS. Show shop_name, category, and location.

Q9. Correlated subquery

For each employee, show their full name, salary, and whether they earn above or below the average salary of their OWN shop. Label this column 'shop_salary_status' (show 'Above Shop Average' or 'Below Shop Average').

This is a correlated subquery - the inner query references the outer query's shop_id.

Q10. HAVING with subquery

Find all shops whose average sale amount is greater than the overall average sale amount across ALL shops. Show shop_name and their average sale amount.

CTEs (Common Table Expressions)

Q1. Basic CTE

Write a CTE called shop_totals that calculates total revenue per shop. Then query it to show the top 5 shops by revenue.

Q2. CTE with filter

Write a CTE called high_earners that finds all employees earning above 60,000. Then join the CTE result to the shops table to show each high earner's shop name and category alongside their name and salary.

Q3. CTE with CASE WHEN

Write a CTE that assigns each sale a tier label based on total_amount:

- **'High Value'** — above 50,000
- **'Medium Value'** — 5,000 to 50,000
- **'Low Value'** — below 5,000

Then query the CTE to count how many sales fall into each tier.

Q4. Two CTEs chained

Write two CTEs:

- gold_customers — all Gold-tier customers
- gold_spending — total amount spent per Gold customer (join to sales)

Then query to show each Gold customer's name and their total spending, ordered highest to lowest.

Q5. CTE for payment method analysis

Using a CTE, calculate the total revenue and number of transactions for each payment method (M-Pesa, Cash, Card). Then show the results with a percentage column showing each method's share of total revenue.

Hint: Use a second CTE or a subquery to get the grand total for the percentage calculation.

Q6. CTE replacing a complex subquery

Rewrite your answer to C5 (total revenue per shop above 50,000) using a CTE instead of a FROM subquery. The result should be identical — this question is about recognising that CTEs and FROM subqueries are often interchangeable.

Q7. CTE with date grouping

Write a CTE that groups all sales by month (use DATE_TRUNC). For each month, show: month, total revenue, and number of transactions. Then query the CTE to show only months with more than 5 transactions.

Q8. CTE + ranking

Write a CTE that calculates total quantity sold per product. Then use ROW_NUMBER() on that CTE result to rank products from best-selling to least-selling. Show: rank, product_name, category, total_qty_sold.

Window Functions

Q1. ROW_NUMBER - overall ranking

Number all employees from 1 to 20 in order of their hire date (oldest first). Show: row_number, full name, shop_name, job_title, hire_date.

Q2. RANK - salary ranking

Rank all employees by salary highest to lowest using RANK(). Show: rank, full name, salary. Notice what happens when two employees have the same salary — RANK skips numbers, DENSE_RANK does not.

Q3. DENSE_RANK - same as E2 but with DENSE_RANK

Repeat E2 using DENSE_RANK instead of RANK. Add a column showing both RANK and DENSE_RANK side by side so the difference is visible. When do the two columns differ?

Q4. PARTITION BY - rank within shop

Rank employees within each shop by salary (highest first). Show: shop_name, full name, salary, shop_salary_rank. Only show rank 1 employees (the top earner per shop).

Hint: Wrap the window function in a subquery or CTE to filter by rank.

Q5. AVG OVER - compare to group average

For each employee, show their salary alongside the average salary of their shop (calculated as a window function, not GROUP BY). Show: full name, shop_name, salary, shop_avg_salary, and the difference (salary − shop average).

Q6. SUM OVER - running total

Show a running total of sale amounts ordered by sale_date. For each sale show: sale_id, sale_date, total_amount, and cumulative_revenue (running sum so far). Order by sale_date, sale_id.

Q7. LAG - previous sale comparison

For each sale (ordered by sale_date), show the current total_amount, the previous sale's amount using LAG(), and the difference between them. Label columns: sale_id, sale_date, total_amount, prev_sale_amount, change.

Q8. LEAD - next sale preview

For each sale (ordered by sale_date), show the current amount and the NEXT sale's amount using LEAD(). If there is no next sale, show 0. Label: sale_id, sale_date, total_amount, next_sale_amount.

Q9. NTILE - customer quartiles

Divide all customers into 4 groups (quartiles) based on when they registered — oldest customers in group 1, newest in group 4. Show: full_name, registered_on, loyalty_tier, and quartile number.

Q10. Combined window function challenge

For each shop, show every sale with: sale_date, total_amount, shop's running revenue total (partitioned by shop_id), and the shop's monthly rank of that sale (rank from highest to lowest within the same month, partitioned by shop and month).

This question combines SUM OVER with PARTITION BY and RANK OVER with PARTITION BY in the same query.

VIEWS

Q1. Create a simple view

Create a view called v_active_shops that shows only active shops (is_active = TRUE) with columns: shop_name, category, location, monthly_rent. Then query the view to show only Food category shops.

Q2. Create a view with JOIN

Create a view called v_employee_directory that shows each employee's:

- full name (first + last concatenated as 'full_name')
- job_title
- shop_name
- category (shop category)
- salary
- manager_name (use COALESCE to show 'No Manager' for managers)

Then query the view to show only Pharmacists.

Q3. Create a view with aggregation

Create a view called v_shop_revenue with: shop_name, category, total_sales (count), total_revenue (sum), avg_sale_value (round to 2dp). Then query it to show shops ordered by total_revenue descending.

Q4. CREATE OR REPLACE - update a view

Replace your v_shop_revenue view to add two new columns: monthly_rent and rent_coverage_ratio = total_revenue / monthly_rent (ROUND to 2dp). The ratio shows how many times over the shop's revenue covers its rent. Query the updated view.

Q5. Create a view with window function

Create a view called v_employee_salary_rank that shows each employee's full name, shop_name, salary, overall salary rank (across all employees), and their rank within their own shop. Query the view to find employees who are rank 1 in their shop but NOT rank 1 overall.

Q6. Create a customer summary view

Create a view called v_customer_summary that shows for each customer: full_name, city, loyalty_tier, total_purchases, total_spent, and a column 'customer_status' that labels them:

- **'VIP'** if total_spent > 100,000
- **'Regular'** if total_spent between 10,000 and 100,000
- **'Occasional'** if total_spent < 10,000 or has made no purchases

Then query the view to count how many customers fall into each status.

Q7. DROP and recreate

Drop the v_active_shops view. Then recreate it as v_shops_full that includes all shops (active and inactive), adding a column 'status' = 'Active' or 'Closed' based on is_active. This tests DROP VIEW IF EXISTS and CREATE OR REPLACE VIEW.

Q8. View on top of a view

Create a view called v_top_shops that queries your existing v_shop_revenue view and shows only shops where total_revenue > 100,000. This demonstrates that views can build on other views.